APCS 程式設計系列 · U09 · 實作中高級 · C++版

堆疊、佇列與內建資料結構 重點講義

stack 後進先出 · 括號配對與後序運算式 · queue 先進先出 · deque · 優先佇列 · map / set 計數去重 · 陣列模擬

1. 堆疊 stack：後進先出（LIFO）

堆疊像一疊盤子：**最後放上去的，最先被拿走**。C++ 用 **stack**：push 放到最上面、top 看最上面、pop 拿掉最上面、empty 問是不是空的。

C++ stack 的三個動作

```cpp
stack<int> st;
for (int x : {1, 2, 3}) st.push(x);
cout << st.top() << " ";   // 3：最後放進去的在最上面
st.pop();                  // 拿掉 3（C++ 的 pop 不回傳值）
cout << st.top() << " " << st.size() << "\n";
```

輸入	輸出
（不需要輸入）	3 2 2

C++ 括號配對：遇到右括號就看堆疊頂端

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    cin >> s;
    stack<char> st;
    bool ok = true;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            char want = c == ')' ? '(' : c == ']' ? '[' : '{';
            if (st.empty() || st.top() != want) { ok = false; break; }
            st.pop();
        }
    }
    if (!st.empty()) ok = false;   // 還有左括號沒配到
    cout << (ok ? "Yes" : "No") << "\n";
}
```

輸入	輸出
([]{()})	Yes

陷阱提醒 括號配對最常漏的兩件事

【錯】堆疊已經空了還去看 top()：C++ 對空的 stack 取 top 是未定義行為，程式可能直接當掉。

【錯】全部讀完就說 Yes：像「((」這種，堆疊裡還剩左括號，一樣不合法。

【對】堆疊的應用：括號配對、後序運算式、復原（上一步）、深度優先搜尋。

2. 後序運算式：用堆疊算，不需要括號

後序運算式把運算符號寫在兩個數字**後面**：3 + 4 寫成 3 4 +。計算規則只有兩條：**讀到數字就放進堆疊；讀到運算符號，就拿出兩個數算完，結果放回去。**

讀到	動作	堆疊內容（左邊是底）
5 1 2	依序放進去	5 1 2
+	拿出 2、1，算 1 + 2 = 3	5 3
4	放進去	5 3 4
*	拿出 4、3，算 3 * 4 = 12	5 12
+	拿出 12、5，算 5 + 12 = 17	17
3 -	放進 3；拿出 3、17，算 17 - 3 = 14	14（答案）

C++ 後序運算式求值

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    stack<long long> st;
    string t;
    while (cin >> t) {                    // 一個一個讀取符號
        if (t == "+" || t == "-" || t == "*") {
            long long b = st.top(); st.pop();   // 先出來的是右邊！
            long long a = st.top(); st.pop();
            if (t == "+") st.push(a + b);
            else if (t == "-") st.push(a - b);
            else st.push(a * b);
        } else st.push(stoll(t));         // 數字直接放進去
    }
    cout << st.top() << "\n";
}
```

輸入	輸出
5 1 2 + 4 * + 3 -	14

陷阱提醒

【錯】拿出來的順序寫反：先拿出來的 b 是**右邊**，要算 a - b。寫成 b - a，減法、除法全錯。

【對】中序轉後序的檢查法：把後序從左到右用上面的規則算一次，看結果跟原式一不一樣。

- 中序 (3 + 4) * 2 的後序是 3 4 + 2 *；前序（符號在前）是 * + 3 4 2。

3. 佇列 queue：先進先出（FIFO）

佇列就是排隊：**先來的先處理**。C++ 用 **queue**：push 排到尾巴、front 看最前面、pop 讓最前面離開。

C++ 丟牌遊戲：丟一張、下一張移到最底

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    queue<int> q;
    for (int i = 1; i <= n; i++) q.push(i);
    while (q.size() > 1) {
        cout << q.front() << " ";   // 最上面那張丟掉
        q.pop();
        q.push(q.front());          // 下一張移到最底下
        q.pop();
    }
    cout << "\n" << q.front() << "\n";
}
```

輸入	輸出
7	1 3 5 7 4 2 6

陷阱提醒 佇列為什麼不能用 list.pop(0)？

【錯】用 vector 的 erase(v.begin()) 拿掉第一個：後面每個元素都要往前搬一格，一次 O(n)。

- 做 n 次就是 $O(n^2)$：$n = 2\times10^5$ 時大約要搬 2×10^{10} 次，一定超時。

【對】queue 與 deque 從前面拿都是 O(1)。

4. deque 與優先佇列

想做的事	C++	Python
堆疊：放 / 看頂端 / 拿	push(x) top() pop()	append(x) st[-1] pop()
佇列：排入 / 看前面 / 拿	push(x) front() pop()	append(x) q[0] popleft()
deque：前面放 / 後面拿	push_front(x) pop_back()	appendleft(x) pop()
優先佇列：放 / 看最優先 / 拿	push(x) top() pop()	heappush(h, x) h[0] heappop(h)
優先佇列預設誰先出？	**大的**先出	**小的**先出

優先佇列不管放進去的順序，每次拿出來的都是「最優先」的那一個，放入、拿出都是 O(log n)。適合「每次都要取目前最大（最小）值」的題目，例如叫號、排程、合併。

C++ priority_queue 與 heapq：誰先出來？

```cpp
priority_queue<int> big;                    // 預設：大的先出
priority_queue<int, vector<int>, greater<int>> small;   // 小的先出
for (int x : {5, 1, 8, 3}) {
    big.push(x);
    small.push(x);
}
cout << big.top() << " " << small.top() << "\n";
```

輸入	輸出
（不需要輸入）	8 1

陷阱提醒

【錯】以為 priority_queue 是小的先出：C++ 預設是**大的**先出，要小的先出得加 greater<int>。
【錯】同分要「先到先出」時，只放分數是不夠的，要把「到達順序」一起放進去比較。

5. map / set / dict / Counter：計數與去重

用途	C++	Python
計數（key → 次數）	map<string, int>：依 key 由小到大	dict 或 Counter：依放入順序
沒出現過的 key	cnt[k]++ 自動從 0 開始	Counter 查到 0；dict 要用 get(k, 0)
去重	set：自動排好序	set：**沒有固定順序**
雜湊版（更快、無序）	unordered_map / unordered_set	（dict、set 本身就是雜湊）

C++ 計數：每個單字出現幾次（依字典序印）

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    map<string, int> cnt;        // map 會依 key 由小到大排好
    string w;
    while (cin >> w) cnt[w]++;   // 沒出現過的 key 自動從 0 開始
    for (auto& [k, v] : cnt) cout << k << " " << v << "\n";
}
```

輸入	輸出
banana apple cherry apple banana apple	apple 3 banana 2 cherry 1

陷阱提醒

【錯】用 unordered_map / unordered_set 走訪輸出：順序不固定，每台電腦可能不一樣，一定要先排序。
【錯】只是想「查有沒有」卻寫 if (cnt[k] > 0)：用 [] 查詢會順便把 k 新增進 map。查有沒有用 cnt.count(k)。
【對】排序條件有兩層（例如票數多的在前、同票依名字）：C++ 把票數取負號放進 pair 排序，Python 用 key=lambda t: (-t[1], t[0])。

6. C 題本：用陣列與 top / front / rear 模擬

APCS 的 C 題本沒有 stack、queue 可用，識讀題會用**陣列加上指標**模擬。重點是看清楚 ++、-- 寫在變數前面還是後面。

C++ C 題本：用陣列模擬堆疊與佇列

```
int st[100], top = 0;              // top = 目前有幾個元素
st[top++] = 4; st[top++] = 7; st[top++] = 1;   // push：先存再加
printf("%d ", st[--top]);          // pop：先減再取 → 1
int q[100], front = 0, rear = 0;   // q[front] ~ q[rear-1] 是佇列
q[rear++] = 4; q[rear++] = 7; q[rear++] = 1;   // 排到尾巴
printf("%d ", q[front++]);         // 從頭拿 → 4
printf("%d %d\n", top, rear - front);         // 各剩幾個
```

輸入	輸出
（不需要輸入）	1 4 2 2

追蹤陣列模擬的四個重點

- **top 從 0 開始**：push 寫 st[top++] = x，pop 寫 st[--top]，top 就是元素個數。
- **top 從 -1 開始**：push 寫 st[++top] = x，pop 寫 st[top--]，top 是最上面那格的索引。
- **佇列**：q[front] 到 q[rear - 1] 是內容，front == rear 代表空的，個數是 rear - front。
- **環狀佇列**：陣列用到底就從頭再用，索引寫成 (rear + 1) % 容量。

讀完這份，你應該要會

- □ 會用堆疊做括號配對，記得檢查「堆疊空了」和「最後還有剩」
- □ 會用堆疊算後序運算式，知道先拿出來的是右邊的數
- □ 會用 queue 模擬排隊，知道 vector 從前面刪除是 O(n)
- □ 知道 priority_queue 預設大的先出，加 greater<int> 變小的先出
- □ 計數用 map / dict / Counter；輸出順序有規定時，一定先排序
- □ 看得懂 C 題本用陣列和 top、front、rear 模擬的堆疊與佇列

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室　　IG @jf_chuang　　LINE 諮詢 jenfoo2021